

TP4 : Des nombres premiers et des images

Dans ce dernier TP, vous mettrez en place ce que vous avez appris précédemment pour proposer une classe permettant de calculer des nombres premiers et trouver la décomposition d'un nombre en produit de nombres premiers. Ensuite, vous verrez ce que représente le concept du broadcasting, et l'utiliserez pour manipuler des images.

1 Des nombres premiers

Dans cet exercice, vous devrez mettre en place une classe **NbPremier**, permettant de faire un certain nombre de calculs basés sur des nombres premiers. Rappelez-vous que l'objectif ici est de proposer un code dans la philosophie "Python", où le temps de développement et la longueur du code doivent rester courts.

Questions

1. Votre classe devra posséder une variable de classe représentant une liste **est_prime** de booléens ainsi qu'une liste **lprime** de nombres entiers :
 - pour tout entier $x \in [0; n]$, x est premier ssi **est_prime(x)** est vrai.
 - pour tout entier $x \in [0; n]$, x est premier ssi $x \in \mathbf{lprime}$.Le constructeur prendra un paramètre n . Au départ, le constructeur ne fera absolument rien (utilisez le mot clef **pass**).
2. A quoi sert, à votre avis, la structure **est_prime** si nous possédons déjà la structure **lprime** ?
3. Proposez une fonction **eratosthene** qui prendra en paramètre une valeur **n**, et initialisera une liste **l** selon le crible d'Eratosthène. Vous trouverez un très bon pseudo-code du crible sur Wikipedia : https://fr.wikipedia.org/wiki/Crible_d%27%C3%89ratosth%C3%A8ne. La compréhension de liste est envisageable quand le calcul relatif à une valeur donnée est "compartimenté" à cette seule valeur. Ici, ce n'est pas le cas : le calcul effectué pour une valeur x va influencer le calcul de tous les multiples de x . Cependant, le slicing sera très utile.
4. Appelez, dans votre constructeur de classe, la fonction précédente. Une fois la structure **est_prime** construite, vous devrez initialiser la structure **lprime**.
5. Proposez une méthode **est_nombre_premier** prenant en paramètre un entier k , et renvoyant un booléen permettant de dire si k est premier ou non. Cette méthode produira une exception de type **ParameterTooHighValue** (à vous de la définir) si $k > n$.
6. La conjecture de Goldbach énonce que tout nombre pair supérieur à 3 est la somme de deux nombres premiers. Proposez une méthode **test_goldbach**, qui affichera, pour tous les nombres pairs p entre 3 et n , deux nombres premiers a et b tels que $p = a + b$. Attention à votre choix de structure de données : choisir la bonne structure de données ici (le dictionnaire) vous permettra d'écrire l'algorithme en quelques lignes seulement grâce aux routines intégrées à Python permettant de simplifier votre travail.
7. Réalisez le rendu de votre travail en suivant les instructions du point 1 en fin d'énoncé.
8. Proposez une méthode **decomposition** prenant un paramètre k , et affichant la décomposition de k en facteurs premiers. Cette méthode émettra une exception si $k > n$. Vous pouvez modifier votre fonction **eratosthene** pour conserver une trace du facteur premier divisant chaque nombre non premier (modifiez dans ce cas les autres algorithmes de votre classe si nécessaire).
9. Réalisez le rendu de votre travail en suivant les instructions du point 2 en fin d'énoncé.

2 Une courte introduction à Numpy et au broadcasting

2.1 Premier contact avec Numpy

La bibliothèque numpy permet de manipuler très simplement, en Python, des matrices. Numpy propose une structure de données **ndarray** que l'on peut se représenter comme les tableaux du C : un accès aux éléments en temps constant, mais pas d'agrandissement ou rétrécissement possible. Je vous invite, lors de ce TP, à bien tester les codes donnés en exemple, pour vous familiariser avec Numpy.

Pour utiliser Numpy, votre programme devra faire

Code Python

```
import numpy
```

Ensuite, vous pouvez utiliser Numpy pour générer des matrices ou des vecteurs. Par exemple, **numpy.random.rand** vous permet de générer des matrices de nombres aléatoires :

Code Python

```
r = numpy.random.rand(3, 7)
print(r)
print(type(r))
```

Sortie

```
[[0.40935811 0.69008816 0.79378602 0.82151288 0.54177539 0.04383064
 0.5253083 ]
 [0.02477785 0.617817 0.82840873 0.2436866 0.9174634 0.32918715
 0.34634032]
 [0.26629371 0.92022695 0.89120653 0.70984127 0.5569866 0.7313052
 0.13194371]]
<class 'numpy.ndarray'>
```

On peut accéder aux éléments d'un **ndarray** comme s'ils s'agissaient d'éléments d'une liste. Il est aussi possible de faire des slices d'une telle structure. Regardez la documentation des fonctions **zeros** et **ones** de la librairie numpy, permettant de générer des tableaux Numpy.

Les tableaux Numpy ont une taille fixe, et "sortir" d'un tableau provoquera une exception :

Code Python

```
r[5,10]=0
```

Sortie

```
Traceback (most recent call last):
  File "test.py", line 7, in <module>
    r[5,10]=0
IndexError: index 5 is out of bounds for axis 0 with size 3
```

On peut récupérer la taille d'un **ndarray** sous forme de tuple grâce à l'attribut **shape** :

Code Python

```
taille = r.shape
print(taille)
print(type(taille))
```

Sortie

```
(3, 7)
<class 'tuple'>
```

2.2 Opérations entre matrices de tailles différentes

Les matrices et vecteurs de Numpy permettent d'utiliser le principe du broadcasting. Pour expliquer ce dernier (qui ne fonctionne qu'avec les matrices Numpy - inutile d'essayer avec les séquences et collections de base de Python), considérons l'exemple suivant :

Code Python

```
import numpy

#On crée deux matrices de nombres aléatoires entre 0 et 9, de taille 5 lignes par 7
#                                colonnes
r = numpy.random.randint(0, 9, [5, 7])
s = numpy.random.randint(0, 9, [5, 7])

f = r+s
```

Ce code fonctionnera comme on pourrait le prévoir, et la matrice **f** contiendra bien la somme des matrices **r** et **s**. Considérons cet exemple légèrement différent :

Code Python

```
import numpy

#On crée deux matrices de nombres aléatoires entre 0 et 9, de tailles différentes
r = numpy.random.randint(0, 9, [3, 8])
s = numpy.random.randint(0, 9, [5, 7])

f = r+s
```

Ici, le programme affiche une erreur :

Sortie

```
Traceback (most recent call last):
  File "test.py", line 7, in <module>
    f = r+s
ValueError: operands could not be broadcast together with shapes (3,8) (5,7)
```

Comme on s'y attendait, les deux matrices n'étant pas de la même taille, l'opération n'a pas pu avoir lieu. Cependant, considérons ce troisième code :

Code Python

```
import numpy

#On crée deux matrices de nombres aléatoires entre 0 et 9, de tailles différentes (
#                                mais compatible pour le broadcasting)
r = numpy.random.randint(0, 9, [1, 7])
s = numpy.random.randint(0, 9, [5, 7])

f = r+s

print("La matrice r vaut")
print(r)
```

```
print()

print("La matrice s vaut")
print(s)
print()

print("La matrice f vaut")
print(f)
print()
```

Sortie

```
La matrice r vaut
[[3 0 7 2 7 6 1]]

La matrice s vaut
[[4 5 8 0 1 2 3]
 [2 0 1 2 1 7 6]
 [5 3 1 4 3 8 2]
 [8 6 1 3 7 3 8]
 [3 4 8 0 0 2 3]]

La matrice f vaut
[[ 7 5 15 2 8 8 4]
 [ 5 0 8 4 8 13 7]
 [ 8 3 8 6 10 14 3]
 [11 6 8 5 14 9 9]
 [ 6 4 15 2 7 8 4]]
```

Ici, malgré que les matrices n'ont pas la même taille, l'opération d'addition a pu être réalisée, et aucun message d'erreur n'est apparu : Numpy a permis cela grâce au broadcasting.

2.2.1 Le broadcasting avec Numpy

Le broadcasting est le fait que, dans certains cas, Numpy peut réaliser certaines opérations entre matrices de tailles différentes, alors que ces opérations sont habituellement réservées à des matrices de même taille. On dira, dans ce cas, que ces matrices ont des **tailles compatibles**.

Décider si le broadcasting est possible Pour savoir si des matrices ont des tailles compatibles, Numpy regarde les dimensions de chaque matrice ayant un rôle dans l'opération. Dans notre exemple, nous avons :

Code Python

```
print(f"La taille de r est {r.shape}")
print(f"La taille de s est {s.shape}")
```

Sortie

```
La taille de r est (1, 7)
La taille de s est (5, 7)
```

Numpy parcourra chaque dimension i de chaque matrice de **droite à gauche** (par rapport à **shape**) et décidera que les matrices ont une taille compatible si, pour chaque dimension :

- Les deux matrices ont la même taille pour la dimension i ou
- L'une des matrices a une taille 1 associée à la dimension i .

Ici, Numpy parcourt les dimensions de chaque matrice de droite à gauche, c'est à dire les éléments des **shape** des deux matrices de la fin au début. Les deux premiers éléments sont égaux (7), Numpy peut passer à la suite. Les deux éléments suivants ne sont pas égaux, mais l'un d'eux est égal à un, donc la comparaison est satisfaisante. Comme il n'y a pas d'autres éléments à parcourir, Numpy considère les deux matrices comme ayant une taille compatible.

Réaliser le broadcasting Pour réaliser l'opération de broadcasting, Numpy va simplement regarder la ou les dimensions des matrices valant 1, et par réplication des données, va rendre la taille de cette dimension égale à la même dimension de l'autre matrice.

Prenons l'exemple précédent de nos deux matrices : la matrice **r** possède 1 ligne et 7 colonnes, tandis que la matrice **s** possède 5 lignes et 7 colonnes. Numpy détecte donc que le nombre de colonnes des deux matrices est égal, et que le nombre de lignes des deux matrices est différent, et que l'une des matrices n'en possède qu'une seule : les tailles sont donc considérées comme compatibles, comme expliqué précédemment.

$$r = \begin{bmatrix} 3 & 0 & 7 & 2 & 7 & 6 & 1 \end{bmatrix} \quad s = \begin{bmatrix} 4 & 5 & 8 & 0 & 1 & 2 & 3 \\ 2 & 0 & 1 & 2 & 1 & 7 & 6 \\ 5 & 3 & 1 & 4 & 3 & 8 & 2 \\ 8 & 6 & 1 & 3 & 7 & 3 & 8 \\ 3 & 4 & 8 & 0 & 0 & 2 & 3 \end{bmatrix}$$

On peut voir le broadcasting comme le processus suivant : Numpy va construire une matrice **r'** en répliquant la matrice **r** de façon à ce qu'elle ait le même nombre de lignes que **s** :

$$r' = \begin{bmatrix} 3 & 0 & 7 & 2 & 7 & 6 & 1 \\ 3 & 0 & 7 & 2 & 7 & 6 & 1 \\ 3 & 0 & 7 & 2 & 7 & 6 & 1 \\ 3 & 0 & 7 & 2 & 7 & 6 & 1 \\ 3 & 0 & 7 & 2 & 7 & 6 & 1 \end{bmatrix} \quad s = \begin{bmatrix} 4 & 5 & 8 & 0 & 1 & 2 & 3 \\ 2 & 0 & 1 & 2 & 1 & 7 & 6 \\ 5 & 3 & 1 & 4 & 3 & 8 & 2 \\ 8 & 6 & 1 & 3 & 7 & 3 & 8 \\ 3 & 4 & 8 & 0 & 0 & 2 & 3 \end{bmatrix}$$

Ensuite, Numpy réalisera l'opération d'addition entre **r'** et **s**, qui ont la même taille. Tout se passe comme si l'opération d'addition avait été généralisée à chaque ligne de **s**.

Le broadcast modifie donc, temporairement, une ou plusieurs dimensions des matrices pour que les tailles des matrices en jeu dans une opération soient égales. Il se peut cependant que les deux matrices d'une même opération soient modifiées. Considérons cet exemple :

Code Python

```
import numpy

#On crée deux matrices de nombres aléatoires entre 0 et 9
r = numpy.random.randint(0, 9, [3, 1])
s = numpy.random.randint(0, 9, [1, 4])

f = r*s

print("La matrice r vaut")
print(r)
print()

print("La matrice s vaut")
print(s)
print()

print("La matrice f vaut")
print(f)
print()
```

Sortie

```
La matrice r vaut
[[4]
 [7]
 [5]]

La matrice s vaut
[[3 4 0 7]]

La matrice f vaut
[[12 16 0 28]
 [21 28 0 49]
 [15 20 0 35]]
```

Notez bien que le signe de multiplication "*" entre deux matrices ne correspond pas, dans Numpy, à la multiplication matricielle, mais à la multiplication élément par élément.

Ici, nous avons :

$$r = \begin{bmatrix} 4 \\ 7 \\ 5 \end{bmatrix} \quad s = \begin{bmatrix} 3 & 4 & 0 & 7 \end{bmatrix}$$

Ici, les deux matrices **r** et **s** n'ont ni le même nombre de colonnes, ni le même nombre de lignes. Cependant, les tailles sont compatibles. Numpy va répliquer le vecteur **r** pour qu'il possède le même nombre de colonnes de **s**, et répliquer le vecteur **s** pour qu'il possède le même nombre de lignes de **r** :

$$r' = \begin{bmatrix} 4 & 4 & 4 & 4 \\ 7 & 7 & 7 & 7 \\ 5 & 5 & 5 & 5 \end{bmatrix} \quad s' = \begin{bmatrix} 3 & 4 & 0 & 7 \\ 3 & 4 & 0 & 7 \\ 3 & 4 & 0 & 7 \end{bmatrix}$$

Ensuite, l'opération de multiplication élément par élément pourra avoir lieu.

Enfin, dans le cas où deux matrices n'ont pas le même nombre de dimensions, Numpy complètera les dimensions manquantes par des 1 "sur la gauche". Prenons l'exemple suivant en considération :

Code Python

```
import numpy

#On crée deux matrices de nombres aléatoires entre 0 et 9
r = numpy.random.randint(0, 9, [5, 7, 3])
s = numpy.random.randint(0, 9, [5, 7])

print(f"La matrice r est de taille {r.shape}")
print(f"La matrice s est de taille {s.shape}")
```

Sortie

```
La matrice r est de taille (5, 7, 3)
La matrice s est de taille (5, 7)
```

Si l'on demande à réaliser une addition entre **r** et **s**, Numpy considérera que les "dimensions manquantes" de la matrice **s**, qui est bidimensionnelle, sont égales à 1 à partir de la gauche. La matrice **s** sera vue comme étant de taille (1, 5, 7). Cette taille n'étant pas compatible avec celle de **r**, aucun broadcasting ne pourra avoir lieu entre ces deux matrices :

Code Python

```
f = r*s

print(f"La matrice f est de taille {f.shape}")
```

Sortie

```
Traceback (most recent call last):
  File "test.py", line 10, in <module>
    f = r*s
ValueError: operands could not be broadcast together with shapes (5,7,3) (5,7)
```

Cependant, dans le cas où **r** serait de taille (3,5,7), alors **s** étant vue comme étant de taille (1,5,7), un broadcast serait possible :

Code Python

```
import numpy

#On crée deux matrices de nombres aléatoires entre 0 et 9
r = numpy.random.randint(0, 9, [3, 5, 7])
s = numpy.random.randint(0, 9, [5, 7])

print(f"La matrice r est de taille {r.shape}")
print(f"La matrice s est de taille {s.shape}")

f = r*s

print(f"La matrice f est de taille {f.shape}")
```

Sortie

```
La matrice r est de taille (3, 5, 7)
La matrice s est de taille (5, 7)
La matrice f est de taille (3, 5, 7)
```

Les nombres étant vus par Numpy comme des matrices de taille 1, il est possible de faire une opération matricielle entre une matrice et un scalaire :

Code Python

```
r = r+1
```

Dans ce cas, grâce au broadcasting, l'opération est effectuée sur chaque élément de la matrice.

3 Jouons avec des images avec Numpy et OpenCV

Dans cet exercice, vous découvrirez comment utiliser la librairie OpenCV (Open Computer Vision) pour lire des images avec Python, et obtenir en retour des tableaux Numpy, puis comment utiliser Numpy pour faire différentes opérations sur ces images.

3.1 Ouvrir des images avec OpenCV

Pour lire des images avec Python, il est conseillé d'importer le module OpenCV, que vous devrez préalablement installer parmi vos librairies sous Pycharm.

Code Python

```
import cv2
```

Ensuite, pour lire une image nommée *chien.jpg* (faisant partie des fichiers du TP) et placée dans le même répertoire que votre script Python, vous ferez :

Code Python

```
im = cv2.imread('chien.jpg')
```

Dans cet exemple, il faudra que le fichier image se situe dans le même dossier que votre script python. Vous pouvez afficher les dimensions de l'image en faisant

Code Python

```
print(im.shape)
```

Sortie

```
(421, 600, 3)
```

L'image possède bien 421 lignes (elle fait donc 421 pixels de haut), 600 colonnes (elle fait donc 600 pixels de large) et, pour chaque pixel, possède trois valeurs. Ces trois valeurs représentent la quantité de bleu, de vert et de rouge de chaque pixel. On peut lire les valeurs de rouge, vert et bleu du pixel de coordonnées (100, 50) en faisant

Code Python

```
import cv2

im = cv2.imread('chien.jpg')
bleu, vert, rouge = im[100, 50]
```

3.2 Afficher des images

Pour afficher des images, vous ferez :

Code Python

```
cv2.imshow('Nom Fenetre', im)
cv2.waitKey(0)
```

Cela fera apparaître une fenêtre avec l'image. L'instruction **waitKey** permet de mettre votre programme en pause jusqu'à ce que vous appuyiez sur une touche de votre clavier. Cela permet, dans le cas d'un affichage réalisé juste avant la fin du programme, d'éviter que le programme ne s'arrête tout de suite et que la fenêtre ne disparaisse avant même qu'elle ne se soit affichée.

3.3 Attention, les images sont typées

Vous devez être prudent lors de la manipulation des tableaux dans Numpy : ces derniers sont typés, contrairement aux variables en Python. Vous pouvez lire le type d'un tableau en faisant :

Code Python

```
print(im.dtype)
```

Sortie

```
uint8
```

L'image est donc vue, ici, comme un tableau Numpy d'entiers 8 bits non signés. Comme dit précédemment, chaque pixel possède trois valeurs (une pour chacune des trois couleurs primaires) et ces valeurs

sont nécessairement entre 0 et 255. On peut s'en convaincre en faisant l'opération suivante (que vous n'avez pas besoin de recopier pour éviter de modifier l'image de départ) :

Code Python

```
print(f"La valeur de bleu du pixel (50, 50) est {im[50, 50, 0]}")
im[50, 50, 0] += 220
print(f"Après y avoir ajouté 220, la valeur de bleu du pixel (50, 50) est {im[50, 50, 0]}")
```

Sortie

```
La valeur de bleu du pixel (50, 50) est 49
Après y avoir ajouté 220, la valeur de bleu du pixel (50, 50) est 13
```

Il faudra donc être attentif, pendant ce TP, à ne pas faire des calculs pouvant risquer un débordement de ces valeurs. Soyez particulièrement attentifs lors des opérations entre des tableaux Numpy, comme l'addition. Les résultats de ces opérations sont stockés dans un tableau Numpy de même type, les débordements sont donc possibles.

Il est donc recommandé, lors d'opérations telles que l'addition entre tableau numpy, de convertir avant ces tableaux en un type plus "grand" pouvant contenir le résultat d'un éventuel débordement. On pourra convertir temporairement un tableau numpy en faisant, par exemple :

Code Python

```
import cv2
import numpy

im = cv2.imread('chien.jpg')

im2 = numpy.uint32(im)

print(im2.dtype)
```

Sortie

```
uint32
```

Sachez cependant que, pour afficher une image, qu'elle soit en couleur ou en noir et blanc, il faut nécessairement que cette dernière soit codée sur des entiers non signés 8 bits. Si vous avez fait, pour des calculs intermédiaires, une conversion d'une image, il faudra vous assurer de faire la conversion en uint8 avant d'afficher votre image.

3.4 Opérations sur les images

Dans cette partie, après avoir lu de nombreuses informations sur la bibliothèque numpy et les images, vous mettrez en œuvre différents algorithmes permettant de faire des modifications sur une image.

Questions

1. Commencez par créer une classe **Image** qui possèdera un constructeur prenant en paramètre un nom de fichier et ouvrira ce fichier dans une variable de classe **im**.
2. Proposez, dans votre classe, une méthode permettant d'afficher l'image à l'écran.
3. Proposez une méthode permettant, dans le cas où votre image serait en couleur, de transformer votre image en noir et blanc. Dans ce cas, les trois "canaux" couleur de l'image doivent être moyennés, et l'image ne doit plus posséder qu'une seule valeur par pixel. Utilisez des opérations matricielles pour faire cette conversion en noir et blanc, et soyez prudents sur les dépassements

de capacité mémoire. Votre méthode ne doit rien faire si l'image est déjà en noir et blanc.

4. Réalisez le rendu de votre travail en suivant les instructions de la partie suivante.
5. Proposez une méthode permettant d'ajouter, à votre image, des bandes horizontales noires tous les n pixels. Vous n'avez pas besoin de boucle explicite dans ce code.
6. Proposez une méthode permettant de calculer les contours de l'image. Cette méthode prendra en entrée un entier k , et calculera, pour tous les pixels p à une distance k ou plus du bord de l'image, la valeur maximum et la valeur minimum de tous les pixels dans un carré de côté $2k + 1$ centré sur p . La différence entre ces deux valeurs sera stockée dans une image temporaire de même taille que l'image de départ, et une fois le calcul terminé, l'image de départ sera remplacée par cette image temporaire.

Attention, cette méthode doit produire une exception si elle est exécutée sur une image couleur, car elle ne doit être utilisée que sur des images en noir et blanc.

4 Rendu de votre travail

Vous devrez rendre votre travail, **sauf le dossier venv**, dans un fichier zip dont le nom aura comme format

Code Python

```
VotreNom_VotrePrenom_Votre_NumeroEtudiant.zip
```

sans espace ni accent.

1. Vous devrez rendre votre programme permettant d'afficher, pour tous les nombres pairs entre 1 et N , les couples de nombres premiers a et b tels que $N = a + b$. Votre programme devra s'exécuter avec la commande :

Code Python

```
python3 goldbach.py N
```

Ce code

Code Python

```
python3 goldbach.py 10
```

devra afficher exactement (respectez la syntaxe, il n'y a par exemple pas d'espace) :

Code Python

```
4=2+2
6=3+3
8=5+3
10=5+5
```

L'affichage de votre programme devra respecter cette syntaxe, et rien d'autre ne devra être affiché sur le terminal.

2. Vous devrez rendre votre programme permettant d'afficher, pour un nombre N , la décomposition en facteurs premiers de N . Votre programme devra s'exécuter avec la commande :

Code Python

```
python3 decomposition.py N
```

Ce code

Code Python

```
python3 decomposition 68
```

devra afficher exactement (respectez la syntaxe, il n'y a par exemple pas d'espace) :

Code Python

```
68=2*2*17
```

Ce code

Code Python

```
python3 decomposition 67
```

devra afficher exactement (respectez la syntaxe, il n'y a par exemple pas d'espace) :

Code Python

```
67=67
```

L'affichage de votre programme devra respecter cette syntaxe, et rien d'autre ne devra être affiché sur le terminal.

3. Vous devrez rendre votre programme permettant de transformer une image couleur en noir et blanc. Votre programme devra s'exécuter avec la commande :

Code Python

```
python3 nb.py entree.jpg sortie.jpg
```

Ce code permettra de transformer l'image **entree.jpg** en une image noir et blanc **sortie.jpg**.

4.1 JARVIS

Pour vous aider dans votre travail, vous pouvez utiliser JARVIS. Pour tester immédiatement votre programme, préparez votre fichier comme expliqué ci-dessus. Puis, envoyez le à l'adresse :

```
chaussard.laga+pytp4_goldbach@gmail.com
```

pour le travail sur la conjecture de Goldbach

```
chaussard.laga+pytp4_decomp@gmail.com
```

pour le travail sur la décomposition en facteurs premiers

```
chaussard.laga+pytp4_nb@gmail.com
```

pour le travail sur les images en noir et blanc.

Deux minutes plus tard, vous devriez recevoir un mail de JARVIS qui aura testé votre programme et vous transmettra les résultats de ses tests. Votre message n'a pas besoin d'avoir de sujet particulier ou bien de message particulier, simplement votre programme en fichier joint. N'incluez qu'un seul fichier avec votre mail (un fichier compressé selon les instructions précédentes).